hw6

my uni: sn3136

harder problems

1.

so we build the chip from two memory chips and each storing 2^k words of 32 bits so chip 0 gonna store the low 32-bit half of each 64-bit word and chip 1 store the high 32-bit half

(a) **inputs to chip 0 and chip 1**

both chips always receive address A on their k-bit address inputs and the data inputs are:

$$\text{Chip0.DataIn} = I_{31:0},$$

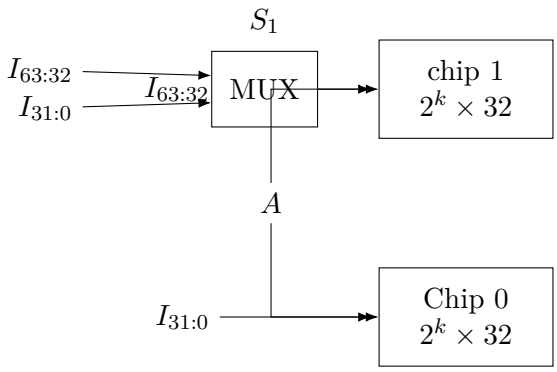
$$\text{Chip1.DataIn} = \begin{cases} I_{63:32}, & S_1 = 1 \\ I_{31:0}, & S_1 = 0. \end{cases}$$

So chip 1's data input can be written with a 2-to-1 MUX:

$$\text{Chip1.DataIn} = \text{MUX}(S_1,\ I_{31:0},\ I_{63:32}),$$

where selector 0 chooses the first input and selector 1 chooses the second input

my diagram:



(b)

chip 0 should be written when either:

- the chip is in 64-bit mode ($S_1 = 1$) or or
- the chip is in 32-bit mode ($S_1 = 0$) and $S_0 = 0$

so

$$W_0 = W\left(S_1 + \overline{S_0}\right)$$

chip 1 should be written when either:

- the chip is in 64-bit mode ($S_1 = 1$), or
- the chip is in 32-bit mode ($S_1 = 0$) and $S_0 = 1$ (upper 32-bit half selected).

si

$$W_1 = W(S_1 + S_0).$$

these can also be written as

$$W_0 = WS_1 + W\overline{S_0}, \qquad W_1 = WS_1 + WS_0$$

(c)

in 64-bit mode ($S_1 = 1$) the output is the concatenation of the two chip outputs so:

$$O_{63:32} = \text{Chip1.Out}, \qquad O_{31:0} = \text{Chip0.Out}$$

in 32-bit mode ($S_1 = 0$) the upper 32 bits must be all zero and the lower 32 bits must come from the selected chip:

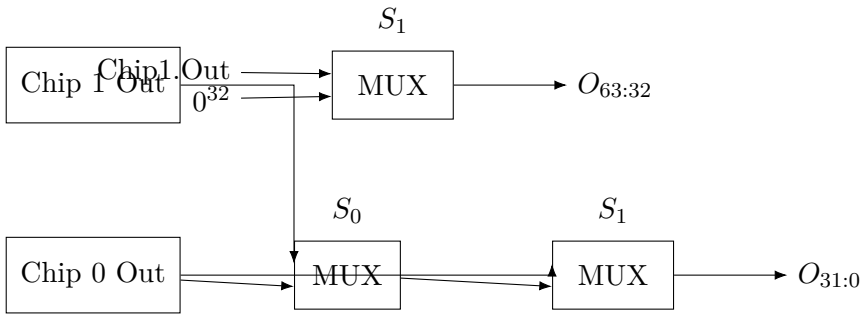
$$O_{63:32} = 0,$$

$$O_{31:0} = \begin{cases} \text{Chip0.Out}, & S_0 = 0 \\ \text{Chip1.Out}, & S_0 = 1 \end{cases}$$

so:

$$O_{63:32} = \begin{cases} 0, & S_1 = 0 \\ \text{Chip1.Out}, & S_1 = 1, \end{cases} \qquad O_{31:0} = \begin{cases} \text{MUX}(S_0, \text{Chip0.Out}, \text{Chip1.Out}), & S_1 = 0 \\ \text{Chip0.Out}, & S_1 = 1 \end{cases}$$

my diagram:



2.

(a)

to clear the whole memory when reset = 1:

- all rows must be enabled
- the data written into every cell must be 0

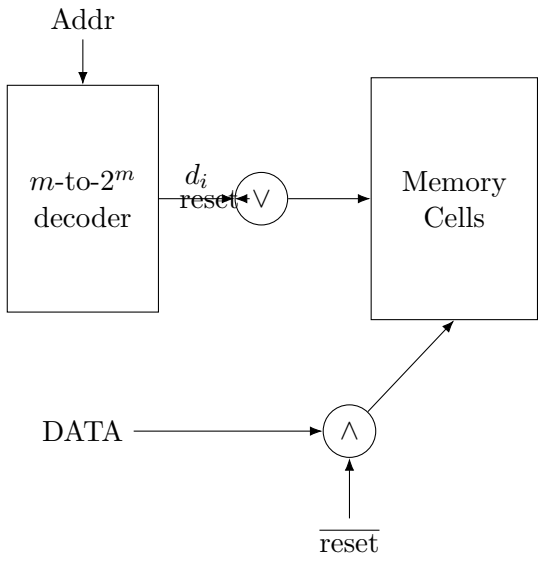
so:

- replace each decoder output d_i by $d_i \vee \text{reset}$;
- replace the data input bus DATA by $\overline{\text{reset}} \cdot \text{DATA}$ bitwise (equivalently, a MUX that selects 0 when reset $= 1$)

then:

- if reset $= 0$, the circuit behaves normally;
- if reset $= 1$, every row is selected and 0 is written everywhere.

A revised diagram is:



(b) **Reset clears only addresses less than the given address.**

Use a 1MASK block on the address. If the address is A, then the 1MASK output is a 2^m-bit vector whose i-th bit is 1 exactly when $A > i$, i.e. exactly for the rows with addresses less than A.

So, during reset:

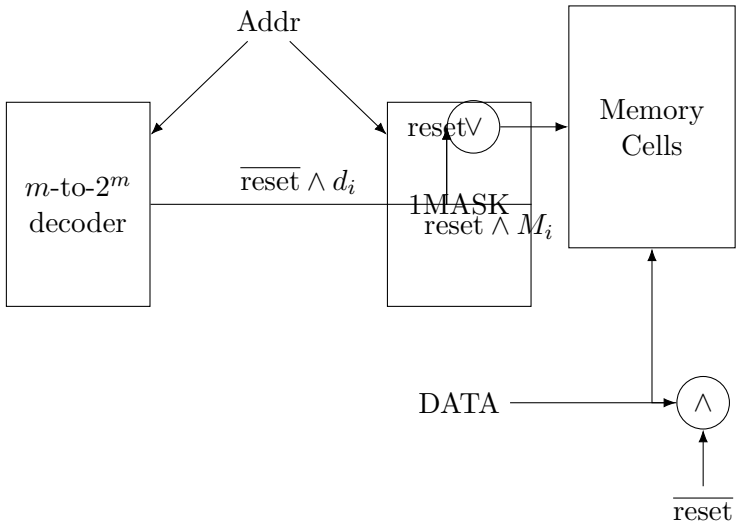
- use the 1MASK output instead of the decoder output,
- force the data bus to 0.

For each row i, the row-enable signal is

$$\text{row}_i = (\overline{\text{reset}} \wedge d_i) \ \vee \ (\text{reset} \wedge M_i),$$

where d_i is the decoder output and M_i is the i-th output of the 1MASK. The data bus is again forced to 0 during reset.

A correct block diagram is:



(c) **Limitation imposed by coincident selection.**

With coincident selection, the address is split into:

$$\text{row bits} = m_1, \qquad \text{column bits} = m_0 = m - m_1.$$

A reset operation selects some set of rows and some set of columns, and therefore clears the *Cartesian product* of those selected rows and columns.

That means the cleared addresses form a rectangular region in the 2D row/column layout. In linear address order, a contiguous interval is therefore possible only in these cases:

1. the entire interval lies within a **single row**; or
2. the interval spans **whole rows**, i.e. it starts at column 0 of its first row and ends at column $2^{m_0} - 1$ of its last row.

So coincident selection **cannot** generally clear an arbitrary contiguous interval $[A, B]$ if that interval starts in the middle of one row and ends in the middle of another row. Once more than one row is selected, the same selected columns appear in every selected row, which would clear non-contiguous addresses in the linear ordering.

3. ALU/register circuit

The selector lines are shared as drawn:

- S_0 controls whether the registers hold or load the final bus.
- S_1 controls the internal operation choices.
- S_2 chooses between the non-adder path and the adder path.

Tracing the circuit gives these internal behaviors:

- If $S_1 = 0$, the non-adder path produces $\text{shl}(A)$ and the adder path produces $A + B$.

- If $S_1 = 1$, the non-adder path produces DATA and the adder path produces $A - \text{DATA}$.
- If $S_2 = 0$, the final bus takes the non-adder path.
- If $S_2 = 1$, the final bus takes the adder path.
- If $S_0 = 0$, both registers hold.
- If $S_0 = 1$, both registers load the final bus.

Here $\text{shl}(A)$ means "shift A left by one bit."

(a) Therefore the 8 input combinations are:

$S_2S_1S_0$	Final bus value	Effect on registers
000	$\text{shl}(A)$	Hold: no state change
001	$\text{shl}(A)$	Load both registers with $\text{shl}(A)$
010	DATA	Hold: no state change
011	DATA	Load both registers with DATA
100	$A + B$	Hold: no state change
101	$A + B$	Load both registers with $A + B$
110	$A - \text{DATA}$	Hold: no state change
111	$A - \text{DATA}$	Load both registers with $A - \text{DATA}$

(b) To transfer the current value in A into B, we need the final bus to equal A, while also loading the registers. Since S_0 must be 1 for loading, we need to find settings that make the final bus exactly A.

From the table above, the only controllable expression that can be forced to equal A for all current values of A is

$$A - \text{DATA},$$

by choosing

$$\text{DATA} = 0.$$

Thus the required setting is

$$S_2 = 1, \qquad S_1 = 1, \qquad S_0 = 1, \qquad \text{DATA} = 0.$$

Under this choice,

$$A - \text{DATA} = A - 0 = A,$$

so after loading, both registers contain A, which means in particular that B receives the value formerly stored in A.